

TwinMOS Corporate Website Platform

Technical Documentation, CMS Administration & Cloud Deployment Guide

Version 1.0 · October 2026

Audience: Webmasters & Software Engineers

System version: API 0.3.0 · 28 public pages · 15 admin modules

Verified: 253 automated tests · 0 security-scan findings

Table of Contents

1. Introduction	1
1.1 Purpose and Audience	1
1.2 System at a Glance.....	1
1.3 Document Conventions	1
1.4 Related Repository Documents	1
2. System Overview	2
2.1 What the Platform Does	2
2.2 The Public Website	2
2.3 Key Public Features.....	3
3. Technology Stack	7
3.1 Stack Summary	8
3.2 Why These Choices.....	8
3.3 Runtime Topology.....	9
4. System Architecture.....	9
4.1 Monorepo Layout	9
4.2 The Content Pipeline (How Admin Changes Reach the Website)	10
4.3 Data Model Overview	10
4.4 Security Model.....	11
5. Admin Console — CMS Management Guide	11
5.1 Dashboard.....	12
5.2 Content Studio (Articles, News, Pages, FAQ).....	13
5.3 Products (Catalog)	14
5.4 Compatibility Matrix (QVL)	15
5.5 Leads & Quotes.....	15
5.6 RMA Board (Returns).....	16
5.7 Careers (Postings & Applications).....	17
5.8 SN-check & Serials (Anti-Counterfeit).....	18
5.9 Partners & Channel.....	18
5.10 Media Library (DAM)	19
5.11 Translation Studio.....	20

5.12 Users & Roles	20
5.13 Audit Log	21
5.14 Settings	21
6. Daily Operations Runbook (Webmaster)	22
6.1 Morning Check (10 minutes)	22
6.2 Publishing Content to the Live Site	22
6.3 Weekly Rhythm	23
6.4 Access Management Rules	23
7. Cloud Deployment Guide (Software Engineer)	23
7.1 Production Topology and Sizing	23
7.2 Step 1 — Provision and Prepare the VM (once)	24
7.3 Step 2 — Database	24
7.4 Step 3 — API Service	24
7.5 Step 4 — Web + Admin Static Bundles	25
7.6 Content Export and Rebuild Cycle	25
7.7 Step 5 — nginx + Cloudflare	25
7.8 Step 6 — Backups and DR	26
7.9 Step 7 — Cutover Checklist	26
7.10 Rollback	26
7.11 Post-Deployment Monitoring	26
8. Testing and Quality Gates	27
8.1 Automated Suites	27
8.2 Running the Gates	27
9. Troubleshooting	30
9.1 Common Issues and Fixes	30
9.2 Escalation Path	31
10. Appendices	31
10.1 Appendix A — Environment Variables Reference (Production)	31
10.2 Appendix B — API Route Map (Public + Auth + Admin)	32
10.3 Appendix C — Role Permission Matrix	33
10.4 Appendix D — Glossary	33
10.5 Appendix E — File Manifest (Deployment-Critical)	34

Note: This Table of Contents is generated via field codes. To ensure page number accuracy after editing, please right-click the TOC and select “Update Field.”

1. Introduction

1.1 Purpose and Audience

This manual is the complete operating guide for the **TwinMOS Corporate Website Platform** — the production content-management and e-commerce support system powering www.twinmos.com. It is written for two roles: the **Webmaster**, who manages day-to-day content, products, leads and support through the admin console, and the **Software Engineer**, who deploys, operates, monitors and troubleshoots the system end to end.

The document covers every administrative module with step-by-step workflows, explains the technology stack and data flows, documents the public website features, and provides a complete cloud-deployment runbook that has been verified against the actual deploy artifacts shipped in the repository (nginx configuration, systemd unit, backup and restore scripts, and production environment templates).

1.2 System at a Glance

Attribute	Value
Product name	TwinMOS Platform (monorepo: twinmos-platform)
Version	0.3.0 (API reported at <code>/api/v1/health</code>)
Public pages	28 routes (~380 EN routes incl. localized)
Admin modules	15 modules across 9 groups
Database tables	36 tables, 4 enums
Automated tests	253 end-to-end + unit tests (26 suites)
Security gates	OWASP-equivalent local scan: 14 check classes, 0 findings
Deployment topology	Cloudflare TLS/WAF → nginx → static web + admin + Node API → PostgreSQL 16

1.3 Document Conventions

- `monospace` — file paths, commands, URLs and configuration keys.
- **Bold** — UI labels, buttons and first-use terms.
- Callout boxes marked **NOTE / IMPORTANT / WARNING** — operational guardrails; the WARNING boxes describe actions that can cause data loss or outage.
- Exhibits are numbered screenshots captured from the live system.

1.4 Related Repository Documents

The repository ships an internal documentation set under `twinmso_codebase/docs/` that this manual complements: the Master Build Plan (00), Architecture (01), API Specification (02), Database Schema (03), Admin/CMS Specification (04), Prototype Migration Guide (05), Admin Console Plan (08), seven phase-evidence files, and operational runbooks (`DEPLOY.md`, `DR.md`, `HYPERCARE.md`) under `docs/runbooks/`. Where this manual summarizes a procedure, the runbook remains the authoritative source of record.

2. System Overview

2.1 What the Platform Does

The platform is a single integrated system with three deployable surfaces sharing one database:

- **Public website** (`apps/web`) — 28 pages: product catalog with faceted shop, product detail pages with warranty and specifications, compatibility finder, anti-counterfeit serial verification, RMA center with public case tracker, newsroom, learn hub, careers with live openings, where-to-buy distributor locator, contact and volume-quote forms. Served as a fully static build — no server-side rendering required.
- **Admin console / CMS** (`apps/admin`) — a React single-page application behind authentication with role-based access control (5 roles), two-factor authentication, and 15 modules covering content, catalog, leads, RMA, partners, careers, media, localization, users, audit and settings. Every mutation writes an audit-trail row.
- **REST API** (`apps/api`) — a Hono service on Node 24 exposing `/api/v1`: authentication (Better Auth), 15 public form types, RMA lifecycle, anti-counterfeit SN-check, i18n bundles, partner portal, and the full admin CRUD surface with Zod-validated contracts and RFC 9457 problem-detail error envelopes.

A deliberate architecture decision (ADR-006): **no external CMS** — the custom admin panel is the CMS, backed by the same schema. Content leaves the database through a build-time export bridge (`cms-content.js`) that the static site merges at runtime, so published changes appear on the website without a server-side render step.

2.2 The Public Website

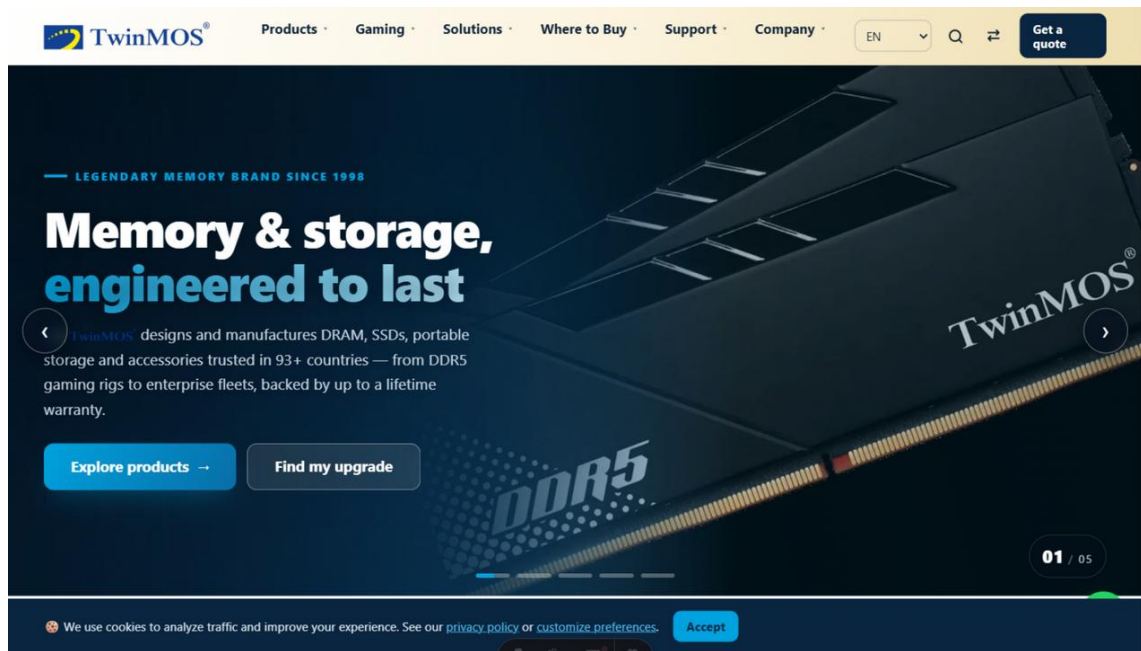


Exhibit 1. Public website home page (dev environment, <http://localhost:4321/>)

The home page presents the brand story, featured product families and entry points to the shop, support center and compatibility finder. Navigation uses a mega-menu organized by product memory (Gaming / Desktop / Notebook DRAM), internal storage (NVMe / SATA SSD) and portable storage, mirroring the catalog taxonomy used throughout the admin console.

2.3 Key Public Features

Product Catalog and Shop

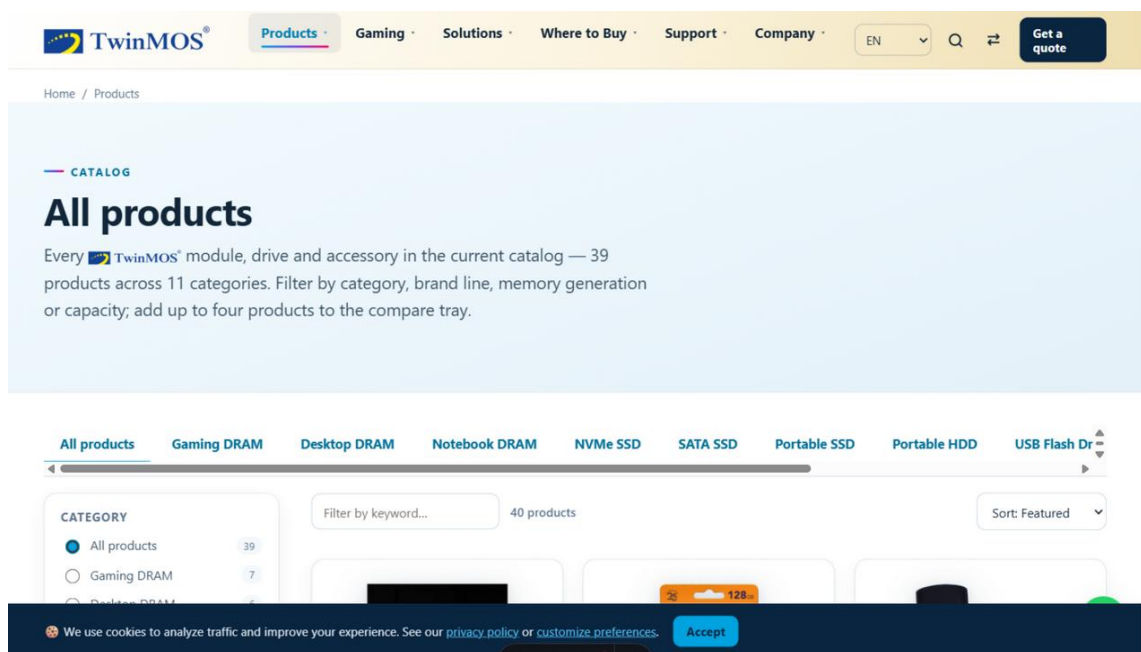


Exhibit 2. Shop page — faceted product grid (40 live products)

The shop renders every published product with category, generation, capacity, interface and form-factor facets inherited from the prototype's filter design. Products managed in the admin console (names, specifications, badges, warranty terms, imagery) flow to this grid through the CMS export bridge — a change made in the admin Products module appears on the site after the next content export (automatic in production builds; see §7 for the export pipeline).

Product Detail with Warranty and Specifications

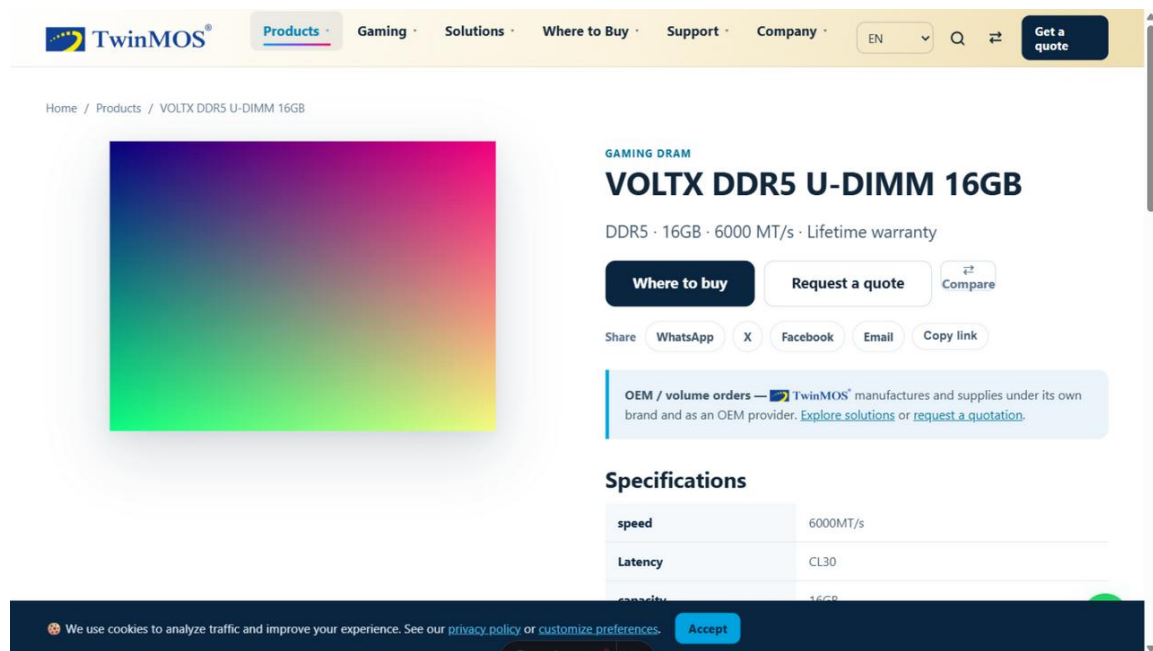


Exhibit 3. Product detail page — CMS-managed warranty chip, short spec line and hero image

Each product page shows the marketing description, the specification table, the warranty term (for example, “Lifetime”), responsive image gallery and variant chips. All of these fields are managed in the admin Products editor; the warranty chip and short-specification line render from CMS data, and the hero image can be served from the API's public media route.

Anti-Counterfeit Serial Verification (Warranty Check)

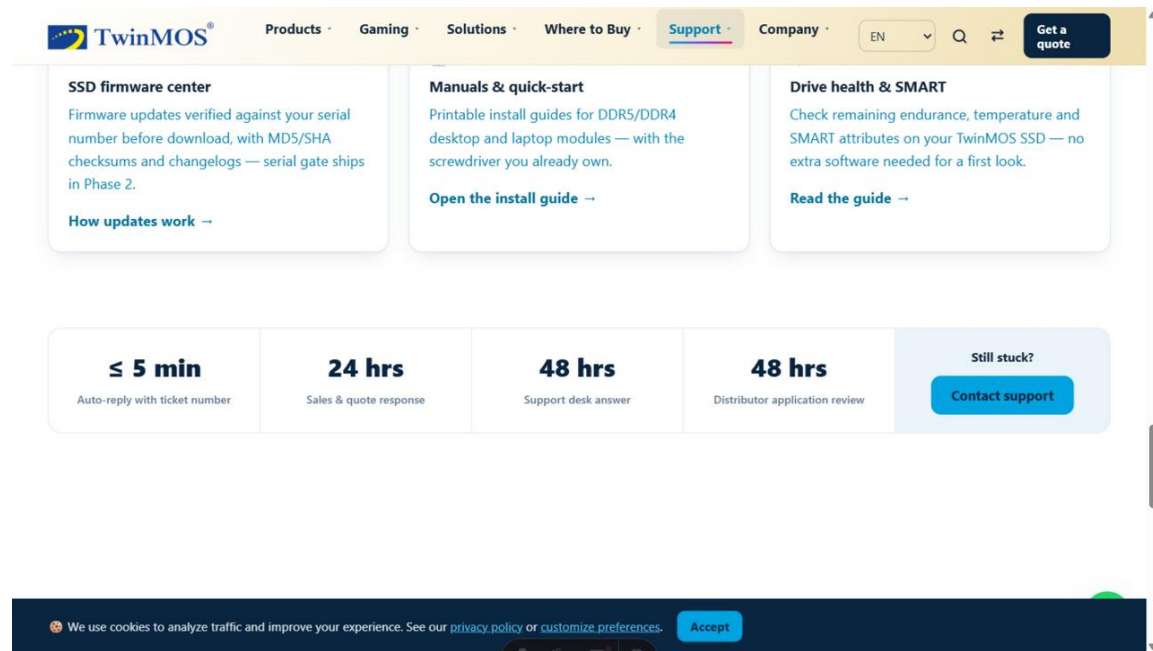


Exhibit 4. Serial-number verification on the Support page — genuine verdict with SKU and manufacture date

On the Support page, customers expand the “Verify a product serial number” card and enter the serial printed on their module. The site calls `POST /api/v1/sn-check`, which looks the serial up in the factory registry, returns a genuine / not-verified verdict with SKU and manufacture date when found, and logs the check (IP, country) for the counterfeit anomaly scan the admin team runs. This is the customer-facing half of the brand-protection workflow described in §5.8.

Compatibility Finder

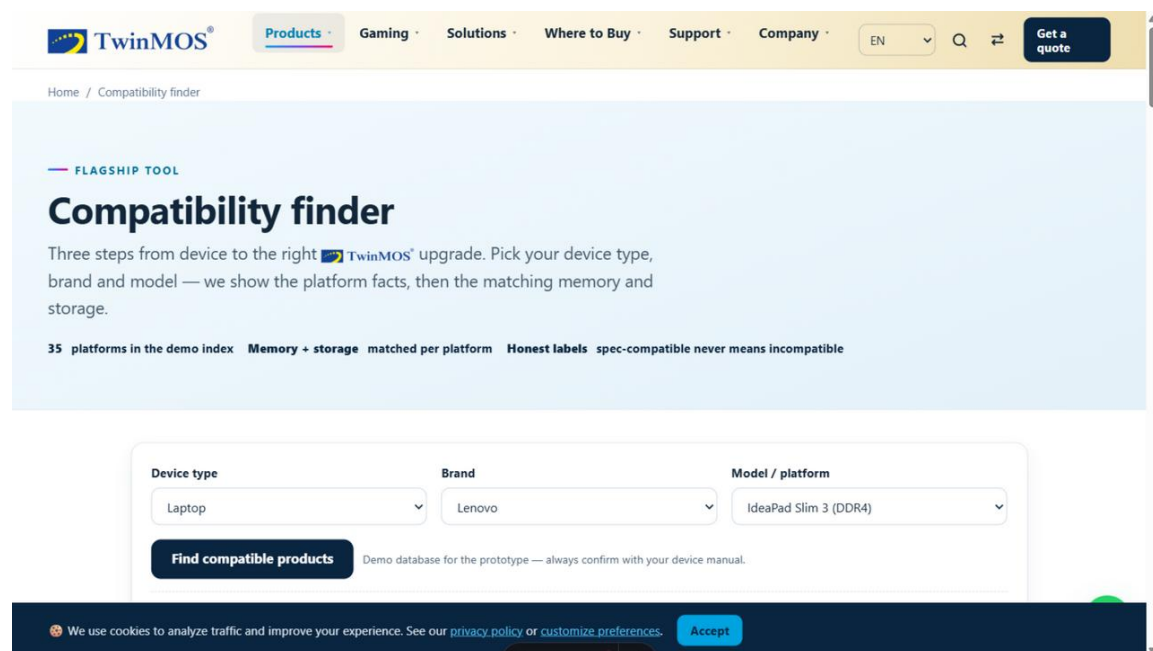


Exhibit 5. Compatibility finder after selecting Laptop → Lenovo → IdeaPad Slim 3 (DDR4)

The compatibility finder walks the customer through device type (Laptop / Desktop PC / Motherboard DIY / Mini PC), brand and model, then renders validated memory guidance — maximum capacity, slot count, supported speed, recommended product categories and the SSD upgrade path. The underlying rules are the QVL matrix maintained in the admin Compatibility module.

Support, RMA and Forms

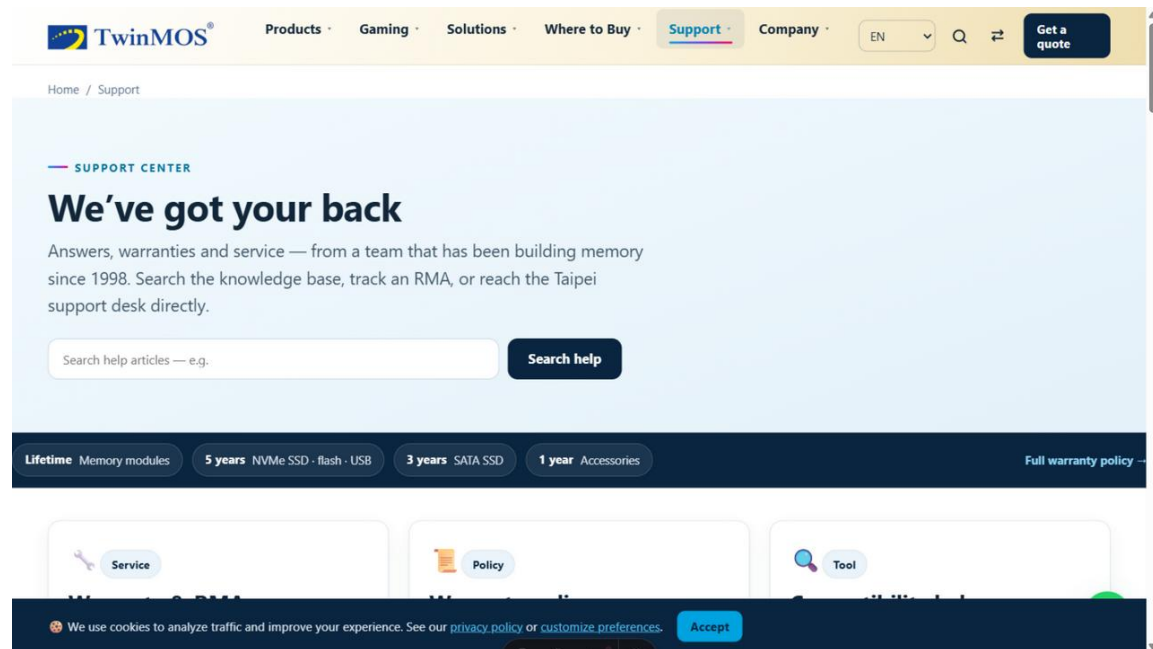


Exhibit 6. Support Center — knowledge base, RMA tracker and the serial-verification card

The RMA Center lets customers submit a return request (which creates a tracked case with an RMA-2026-xxxxxx reference) and follow its status through the seven-state pipeline. Fifteen public form types (contact, quote, RMA, distributor application, job application, counterfeit report, and others) post to the API with mandatory privacy consent and Cloudflare Turnstile anti-spam protection in production.

Careers and Where to Buy

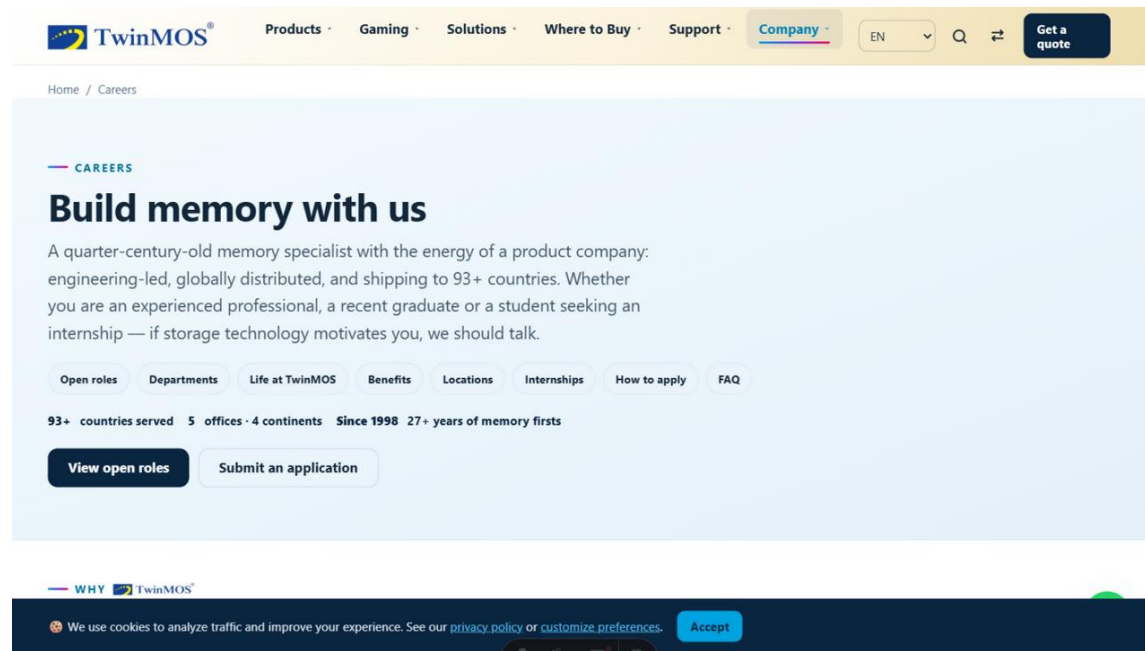


Exhibit 7. Careers page — openings table including a CMS-published role with apply-by date

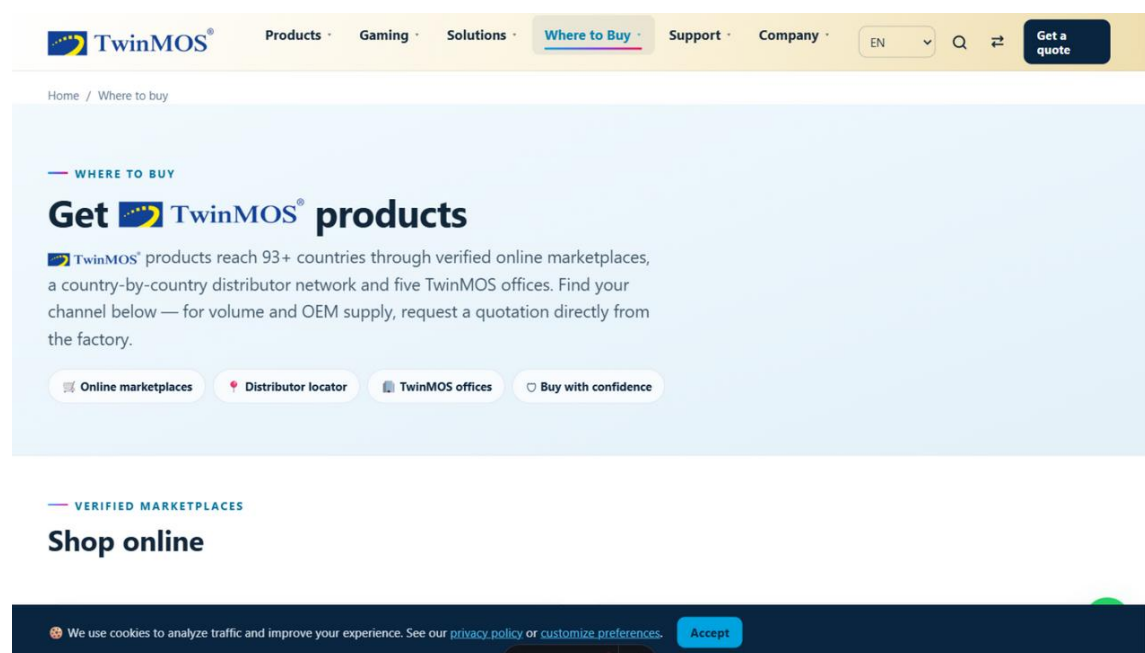


Exhibit 8. Where to Buy — country distributor locator with region filter chips

The careers page lists published job postings (title, location, apply-by deadline, one-line summary and an apply link) — rows managed in the admin Careers module. The Where-to-Buy page is a country-by-country distributor locator with search, region chips and channel-status filters, fed by the directory the admin Partners module maintains; verified online marketplaces are listed in a parallel section.

3. Technology Stack

3.1 Stack Summary

Layer	Technology	Version	Notes
Web framework	Astro (static output)	7.3.5	MPA matching the audited prototype 1:1
Admin framework	React SPA (Vite + TanStack Query)	React 19.3 / Vite 8.3	Code-split per module; tabbed workspace shell
API	Hono on Node 24	Hono 4.13	TypeScript executed natively via --experimental-strip-types
ORM / DB access	Drizzle ORM	0.45.3	Parameter-bound query builders only; SQL via tagged templates
Dev database	PGLite (embedded Postgres)	0.5.8	Dev/test only — the API refuses it under NODE_ENV=production
Production DB	PostgreSQL 16	—	Same schema, zero rewrite
Authentication	Better Auth	1.7.6	Sessions, TOTP MFA, roles, admin plugin
Validation	Zod shared contracts	4.6.5	One schema validates API input and admin forms
Site search	Pagefind	1.5.2	Build-time fuzzy index in the static bundle
Email	Resend	6.30.0	Transactional; dev driver writes JSON to an outbox
Image processing	Sharp	0.35.5	Responsive WebP/AVIF derivatives per upload
Web styling	Prototype main.css design tokens	—	No CSS framework on the public site — the audited CSS is the contract
Testing	Vitest + Playwright + axe-core + Lighthouse CI	—	253 automated tests; parity and accessibility gates

3.2 Why These Choices

- **Astro static output** — the public site ships as pure static files behind a CDN: fastest possible page loads, zero server surface to patch, and pixel parity with the approved prototype is enforced by screenshot-diff gates.
- **Embedded PGLite in dev, real Postgres in production** — a single developer command boots the full stack with a real Postgres dialect in-process, while production connects over the wire with no code changes. The API hard-refuses PGLite when NODE_ENV=production.

- **Better Auth with TOTP MFA** — production-grade session management, hardware-independent two-factor enrollment with QR code and backup codes, and an admin plugin providing user ban/session-revocation primitives.
- **Zod contracts shared across all three apps** — one schema definition validates the API route, the admin form and the public payload, eliminating the classic three-apps-three-validations drift.
- **No external CMS (ADR-006)** — the admin panel is the CMS. This keeps content, catalog and support workflows in one audit-trailed system instead of syncing to a third-party service.

3.3 Runtime Topology

Surface	Dev URL	Production URL	Served by
Public website	http://localhost:4321/	https://www.twinmos.com/	nginx static files (CDN-cached)
Admin console	http://localhost:5173/	https://www.twinmos.com/admin/	nginx static SPA shell
API	http://127.0.0.1:8787/api/v1/	https://www.twinmos.com/api/v1/	Node systemd service behind nginx /api/ proxy
Database	./apps/api/data/dev.pgdata (PGLite)	PostgreSQL 16 (same VM or dedicated)	—

NOTE — In development the three surfaces run on separate ports and the site talks to the API cross-origin (`PUBLIC_API_URL + ALLOWED_ORIGIN` in `apps/web/.env` and the root `.env`). In production everything is same-origin behind one nginx server block, which is why the production templates leave those cross-origin settings unset.

4. System Architecture

4.1 Monorepo Layout

```

twinmso_codebase/
├─ apps/
│   ├─ web/      — Astro public site (28 pages)
│   ├─ admin/    — React 19 admin SPA (15 modules)
│   └─ api/      — Hono REST API + scripts (migrate/seed/reset)
├─ packages/
│   ├─ db/       — Drizzle schema (36 tables), migrations, PGLite client
│   └─ shared/   — Zod contracts, enums, RBAC matrix
├─ tooling/      — importers/exporters, DR drill, OWASP-equivalent scan
└─ deploy/       — nginx.conf, systemd unit, backup/restore scripts

```

└ docs/

— master plan, specs, ADRs, phase evidence, runbooks

4.2 The Content Pipeline (How Admin Changes Reach the Website)

Because the public site is static, published content moves through an explicit export-and-merge bridge rather than server-side rendering:

1. An editor publishes content in the admin console (product, article, job posting, compatibility rule, distributor entry). The API writes the row and an audit-trail entry.
2. The build-time exporter (`tooling/export-content.mjs`) queries the database for published-and-due records and regenerates `apps/web/public/assets/js/cms-content.js` — a small static data file shipped with the site.
3. At page load, `cms-merge.js` merges that file into the page data before rendering: CMS products override matching prototype entries field-by-field, new SKUs append to the shop grid, job rows join the careers openings table, compatibility rules merge into the finder hierarchy, and distributor cards join the Where-to-Buy grid — always deduplicated so nothing renders twice.
4. In production this export runs automatically in the web build's prebuild step; in development the operator re-runs the exporter after catalog edits (the exact command is in §7.6).

NOTE — The bridge is field-aware: an empty CMS field never overwrites prototype data, so day-one rendering is pixel-identical to the approved design. The master plan's long-term direction is for the static prototype data to retire entirely once every entity is CMS-managed.

4.3 Data Model Overview

The schema (`packages/db/src/schema.ts`, 36 tables) groups into eight domains:

Domain	Tables	Purpose
Identity & audit	user, session, account, two_factor, verification, audit_log	Better Auth tables plus MFA and the 12-month audit trail
Content	article, news_post, page, faq, content_revision, content_comment	CMS entities with 5-state workflow, revision snapshots and review comments
Catalog	brand, category, product, product_variant, compatibility_rule	Products with pricing, warranty, facets; QVL matrix
Channel	distributor, marketplace_listing, partner_org, partner_member, partner_asset	Where-to-Buy directory + gated partner portal
Support	form_submission, form_note, rma_request, rma_event	15 lead types with SLA; 7-state RMA pipeline with event log

Domain	Tables	Purpose
Brand protection	serial_registry, sn_check	Factory serials + public verification log for anomaly scans
Careers & i18n	job_posting, job_application, locale, translation	Openings/applications; 9-locale string workflow
Media & settings	media_asset, media_folder, setting, redirect	DAM with responsive variants; site settings and redirects

4.4 Security Model

- **Authentication** — Better Auth email/password with server-enforced 10-character minimum (dev seed may use shorter throwaway passwords via the offline setter script). TOTP two-factor with QR enrollment, 10 backup codes, and lockout-aware verification. Sessions are 8 hours, refreshed every 30 minutes.
- **Authorization** — five roles (super_admin, admin, editor, author, viewer) enforced server-side on every route; the admin sidebar filters modules by role. Governance invariants (at least one active super_admin) are protected against concurrent demotion/ban races.
- **Audit trail** — every mutation writes an audit_log row (actor, action, entity, diff, request id, IP) with 12-month retention; the Audit Log module provides filterable review.
- **Input validation** — Zod schemas on every write; RFC 9457 problem+json error envelopes; upload allow-lists with magic-byte sniffing and SVG active-content rejection.
- **Rate limiting & anti-spam** — per-IP buckets on public endpoints (10 SN-checks/minute), Turnstile on public forms (bypass flag exists for dev only and is hard-refused in production), and TRUST_PROXY handling for Cloudflare client IPs.
- **Transport & headers** — TLS via Cloudflare (full-strict), CSP allowing only challenges.cloudflare.com externally, HSTS, X-Frame-Options DENY, nosniff, referrer and permissions policies set at nginx.

5. Admin Console — CMS Management Guide

This section walks every module of the admin console. The console is reached at /admin on the production domain (<http://localhost:5173/> in development) and requires a staff account. Sign-in offers optional TOTP two-factor; staff without MFA receive a skippable enrollment prompt. The workspace uses a grouped sidebar, a mega-menu (Modules button), multi-tab editing and a Ctrl/Cmd+K global search palette that queries leads, products, content, RMA and media at once.

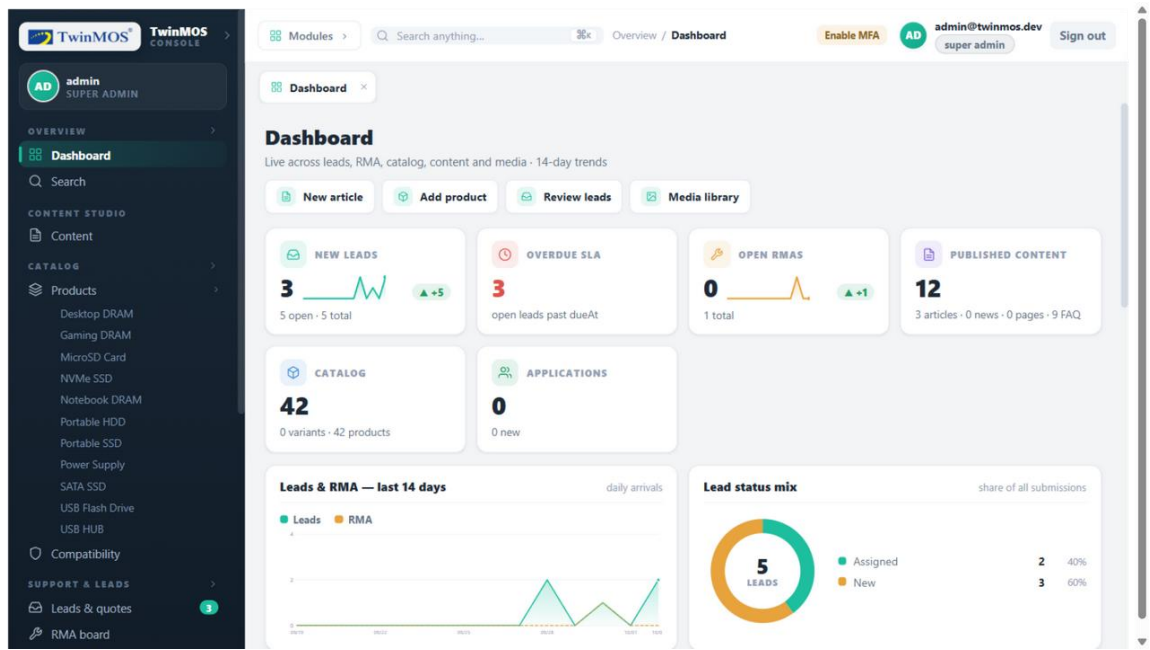


Exhibit 9. Admin console sign-in (split-screen brand rail, TOTP-aware)

NOTE — Roles gate everything: viewers read; authors write content drafts; editors publish and manage channel data; admins manage orgs; super_admins manage users, locales and governance. Buttons for actions your role lacks are hidden or disabled server-side as well as in the UI.

5.1 Dashboard

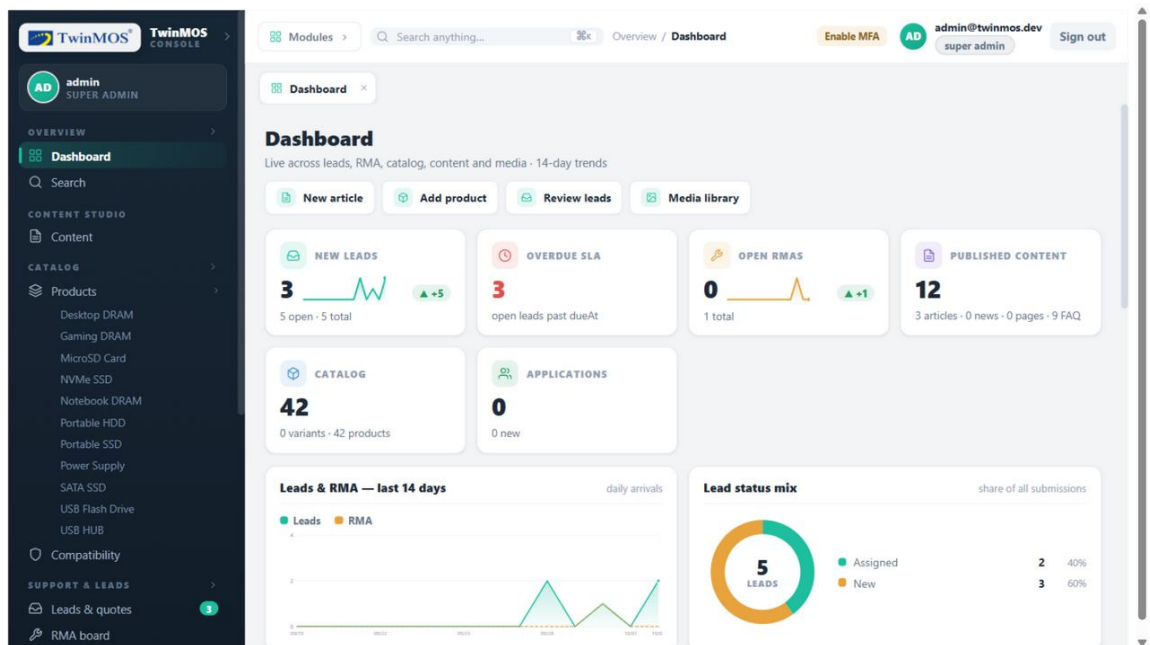


Exhibit 10. Dashboard — KPI strip, 14-day trends, SLA risk queue and activity feed

The dashboard is the morning cockpit: live counts of new leads, overdue SLA, open RMAs, published content, catalog size and applications; a 14-day leads-and-RMA chart; lead status mix and by-type

split; an SLA risk queue (oldest due first — click any row to open the lead); media alt-text compliance; and the latest audited mutations. Four quick actions jump straight into common tasks (New article, Add product, Review leads, Media library).

5.2 Content Studio (Articles, News, Pages, FAQ)

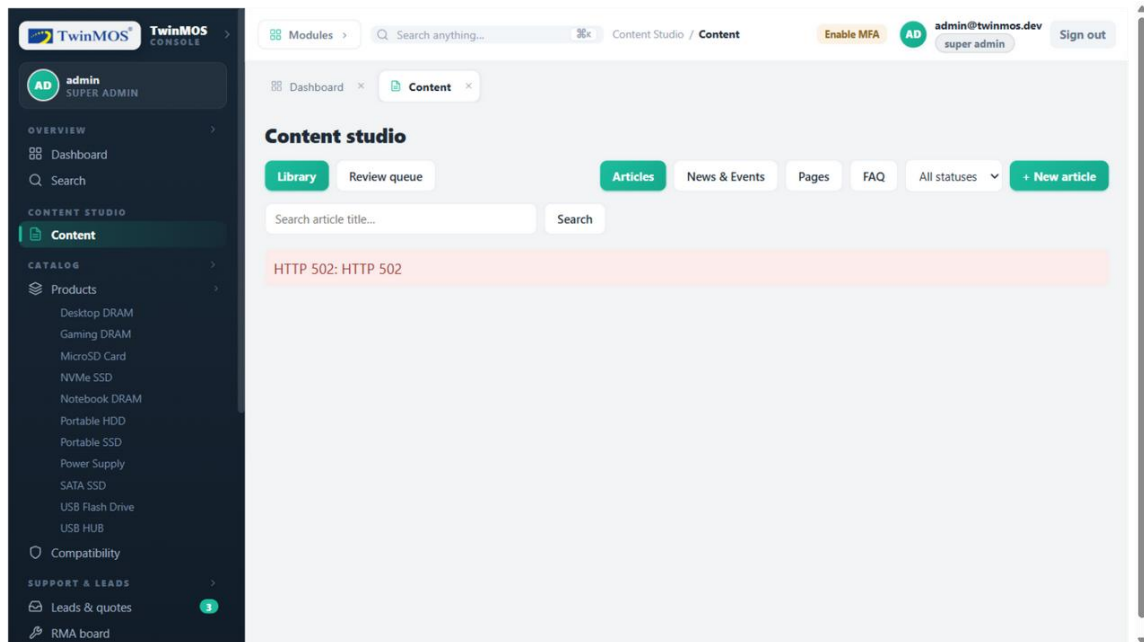


Exhibit 11. Content Studio — library with entity tabs, status filter, bulk publish and pagination

What it manages

- **Articles** — knowledge-base and learn-hub pieces (markdown body with a formatting toolbar and direct media-library image insertion).
- **News & Events** — newsroom posts with tag and optional event date.
- **Pages** — composed visually from 13 registered block types via the page builder (drag-reorder, live desktop/tablet/mobile preview, JSON escape hatch).
- **FAQ** — question/answer entries grouped by area (support-facing).

Step-by-step: publish an article

1. Open **Content Studio** and press **+ New article**.
2. Enter the title (slug auto-derives), a deck/summary, and write the body in markdown. A live word count and reading-time estimate appear under the editor.
3. Press **Create draft**. The editor remounts on the saved row and workflow buttons appear.
4. Choose **Submit for review** (author) or **Publish** (editor+). Optionally set a future date with Publish at to schedule — the scheduled publisher promotes it automatically.

- Published articles appear on the website's news/learn surfaces after the content export (automatic in production builds). The Review Queue tab shows items awaiting an editor: open one to see a visual diff against the published revision, leave comments, then publish or send back.

Revisions, rollback and conflicts

Every publish snapshots the prior version. The Revisions tab lists snapshots with actor and time, shows a diff against your current draft, and offers one-click Rollback (editor+). Saves carry optimistic locking: if a colleague saved while you were editing, the API refuses yours with a conflict banner — reload fresh and re-apply, nothing is silently overwritten.

5.3 Products (Catalog)

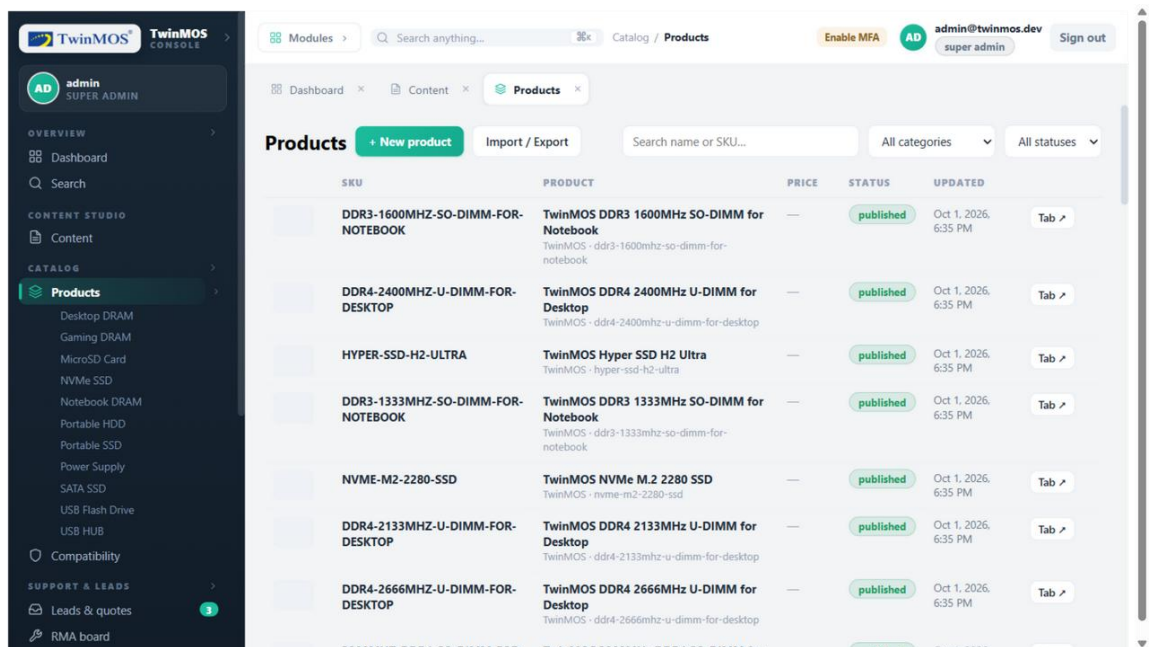


Exhibit 12. Products — searchable catalog with pricing, status, facets and taxonomy filter

Step-by-step: create a product

- Open **Products** and press **+ New product**.
- Fill **Identity**: name, SKU (A-Z 0-9 dashes), slug, status, brand, category.
- Complete **Storefront card**: the catalog card line (one-liner under the product name), marketing description and badges. A live preview mirrors the public shop card as you type — including a live-on-next-export / draft indicator.
- Set **Shop facets**: generation, capacity, interface and form factor. These drive the public shop filters; suggestions are built-in.

5. Complete **Pricing & warranty** (list price + authorized currency, warranty term shown on the product page), **Specifications** (key/value rows), **Datasheets** and **Images** (hero + gallery via the media picker).
6. Press **Create product**, then **Publish** when ready. Published products flow to the public shop/product pages on the next content export.

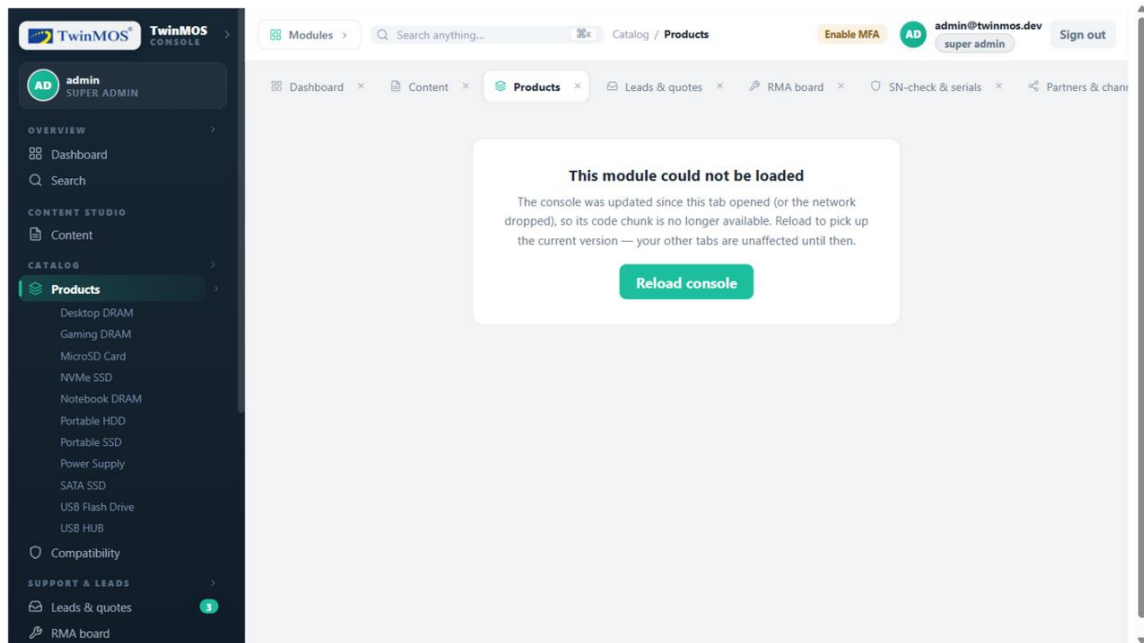


Exhibit 13. Product editor — storefront preview, facets, pricing & warranty

Variants, import and export

The Variants & SKUs tab manages hardware variants (capacity/speed/finish/lighting with per-variant price and stock) inside the product's designed extension point. The Import / Export dialog downloads a CSV template and uploads batch catalogs with dry-run validation; a full-catalog export is one click. Product deletions are soft (recoverable) and blocked while media assets reference the product.

5.4 Compatibility Matrix (QVL)

The Compatibility module maintains the validated-device rules behind the public finder: device type tab, brand, model, memory generation, form factor, maximum RAM, slot count, validated speed, recommended product categories and the SSD upgrade note. Search, type/generation filters and coverage counters sit above the list; batch CSV import (dry-run first, upsert by type+brand+model+gen+form) handles factory QVL manifests. Every rule change is editor-guarded and audited.

5.5 Leads & Quotes

Leads & quotes

Analytics — 5 leads (0 spam) · 5 open · 3 overdue SLA · avg first reply 70h · 4 awaiting reply

Search email, ref, content... All types All statuses Any priority Any SLA Anyone

Reset Export CSV

5 leads matching the current filters

REF	TYPE	FROM	PRIORITY	SLA	REPLY	STATUS	AGE	
FB-2026-5deceef1	feedback	xss@example.com	normal	on track	waiting 0h	new	22m ago	unassigned
CT-2026-dcd45e89	contact	a@b.io	normal	on track	waiting 1h	new	34m ago	unassigned
RMA-2026-b2b17b4c	rma	qa@twinmos.dev	normal	Δ overdue	waiting 37h	new	37h ago	unassigned
TS-2026-DEMO0002	support ticket	field@mesa-systems.example	urgent	Δ overdue	waiting 76h	assigned	3d ago	unassigned
QT-2026-DEMO0001	quote	procurement@nordicpc.example	high	Δ overdue	↔ 70h	assigned	3d ago	admin

« Latest Older »

Exhibit 14. Leads & quotes — SLA-tracked inbox with ownership routing and speed-to-lead column

The lead workflow

1. A public form (quote, contact, support ticket, distributor application...) posts to the API; the submission lands in the inbox with status **new**, a reference code (QT-/TS-/RMA-2026-...), priority and a first-response SLA deadline derived from its type.
2. Filter the inbox by type/status/priority/SLA, ownership (Anyone / Unassigned queue / Mine / a specific owner) or free text across email, reference and payload.
3. Open a lead to see the full payload, SLA state and the pipeline strip (new → assigned → in progress → resolved / closed / spam). Reassign via the owner dropdown or take it yourself.
4. Use **Reply to customer** to email the enquirer directly from the case — the mailer sends, first-response time is stamped, the lead is claimed if unowned, and the reply is logged as an internal note. Speed-to-lead metrics (average first reply, share answered within an hour, currently-waiting count) appear in the analytics strip and as a per-row Reply column.
5. Add internal notes for collaboration (audited), then advance the status. Everything on the lead is audit-trailed.

Reading the analytics

The collapsible strip above the inbox shows the cumulative funnel, SLA gauges (overdue / due soon / healthy), arrivals by type and the reply-rate gauge. The dashboard's SLA risk queue mirrors the most urgent rows.

5.6 RMA Board (Returns)

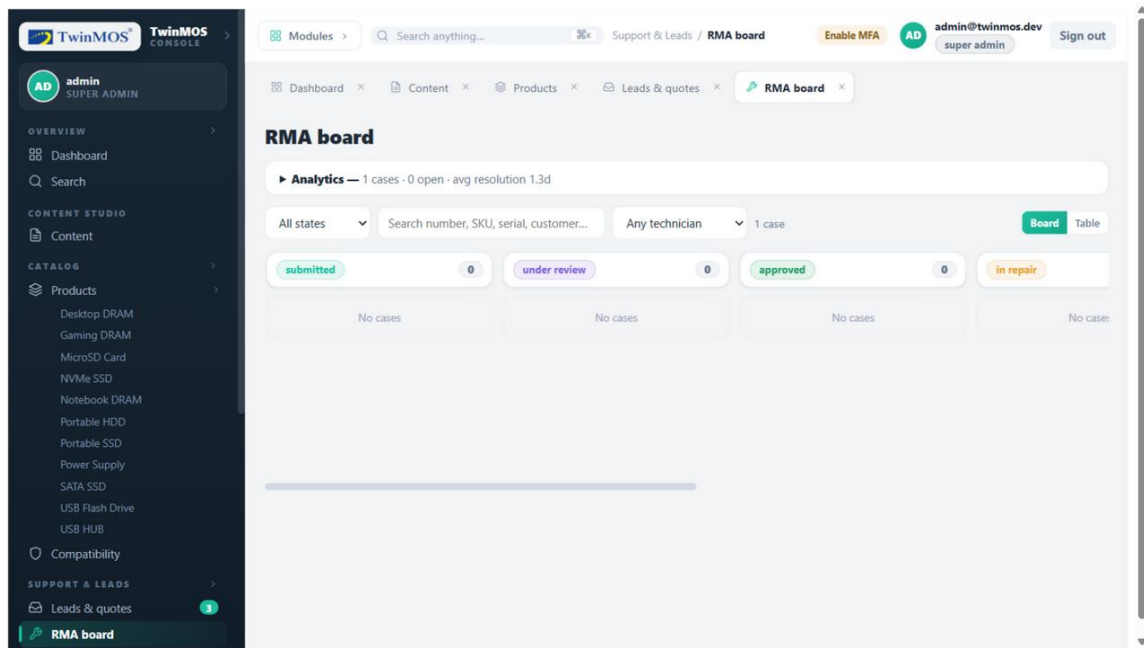


Exhibit 15. RMA board — 7-column Kanban with optimistic transitions and assignee chips

Working a case

1. Cases arrive from the public RMA form as **submitted**. The board defaults to a Kanban view — one column per state, oldest first, with card chips for age, SKU, serial and technician.
2. Move a case with the arrow buttons on its card (optimistic — the card jumps instantly and rolls back on server refusal). The detail panel offers the same transitions with a note and an **Email customer** checkbox that sends a status update through the mailer.
3. Open a case to see the customer, issue, warranty tier, the **serial authenticity check** card (verdict verified / not-in-registry / suspicious with registry SKU match and 24-hour distinct-source activity — the verify-before-service gate), the timeline of every transition, the assignment dropdown and a free-form **Reply to customer** composer.
4. Assign a technician; the chip follows the case across board and detail views.

NOTE — The serial-authenticity card is the operational half of brand protection: it joins the RMA's serial to the factory registry and the last 24 hours of public SN-checks using the same >5-distinct-sources rule as the anomaly scan, so warranty service is never approved on a leaked or counterfeit serial without the team seeing it.

5.7 Careers (Postings & Applications)

The Careers module manages job postings (department, location, type, seniority, apply-by deadline with red/amber closing warnings, salary band where legally required, equal-opportunity statement, markdown description whose first paragraph ships to the public careers page as the summary) and the applications inbox (pipeline new → in review → resolved, per-role filter, candidate drill-down with the full payload and a one-click reply that emails the candidate and moves new applications

into review). Analytics strip: live postings, new applications, top roles by applicant count, 14-day arrivals.

5.8 SN-check & Serials (Anti-Counterfeit)

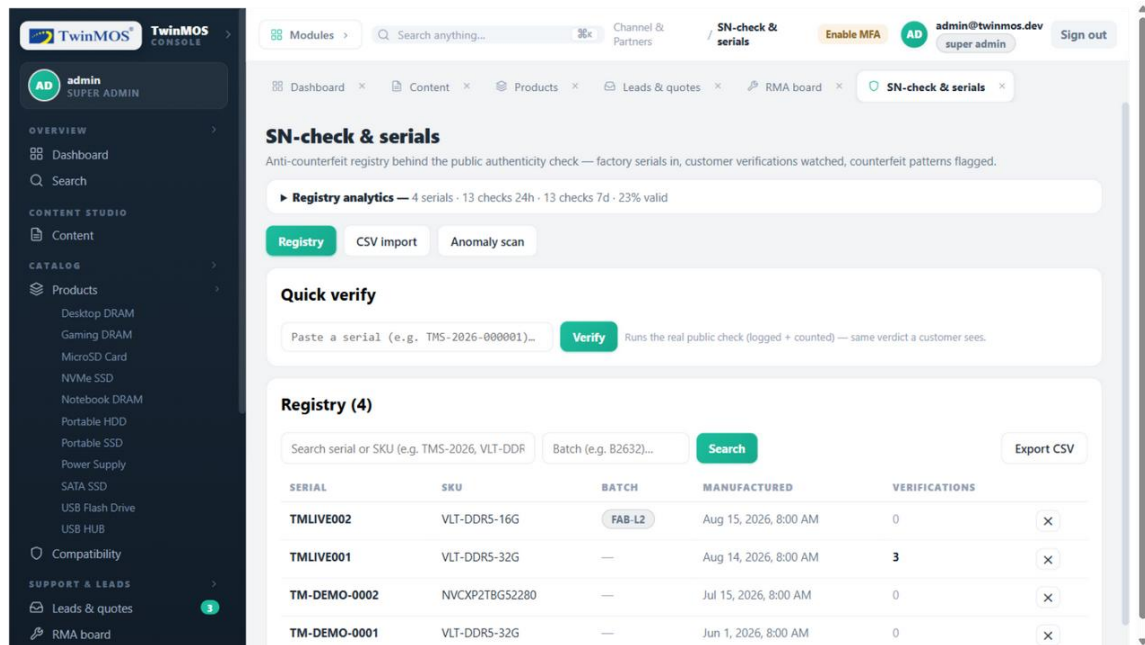


Exhibit 16. SN-check & serials — registry analytics, quick verify, drill-down and import

The brand-protection loop

1. Import factory serials: **CSV import** accepts production-run manifests (header serial,sku,manufacturedAt,batch) with a mandatory dry-run row report; rows upsert by serial. File upload or paste both work.
2. Monitor the registry: the analytics strip tracks registry size, checks in 24h/7d, valid rate, top SKUs and countries by verification volume. **Quick verify** runs the real public check on any serial without leaving the console.
3. Investigate a serial: click any row (or Drill down from quick verify) for the registry data, 24-hour distinct-source activity and the full verification log with verdict (verified / unknown / suspicious).
4. Run the **Anomaly scan** for serials verified from more than N distinct IPs or countries in 24 hours — the distribution signature of leaked or counterfeit serials — and email the ops alert when the list is non-empty.
5. Registry lifecycle: delete a serial (editor+) when it must stop verifying; the verification history is preserved for investigation.

5.9 Partners & Channel

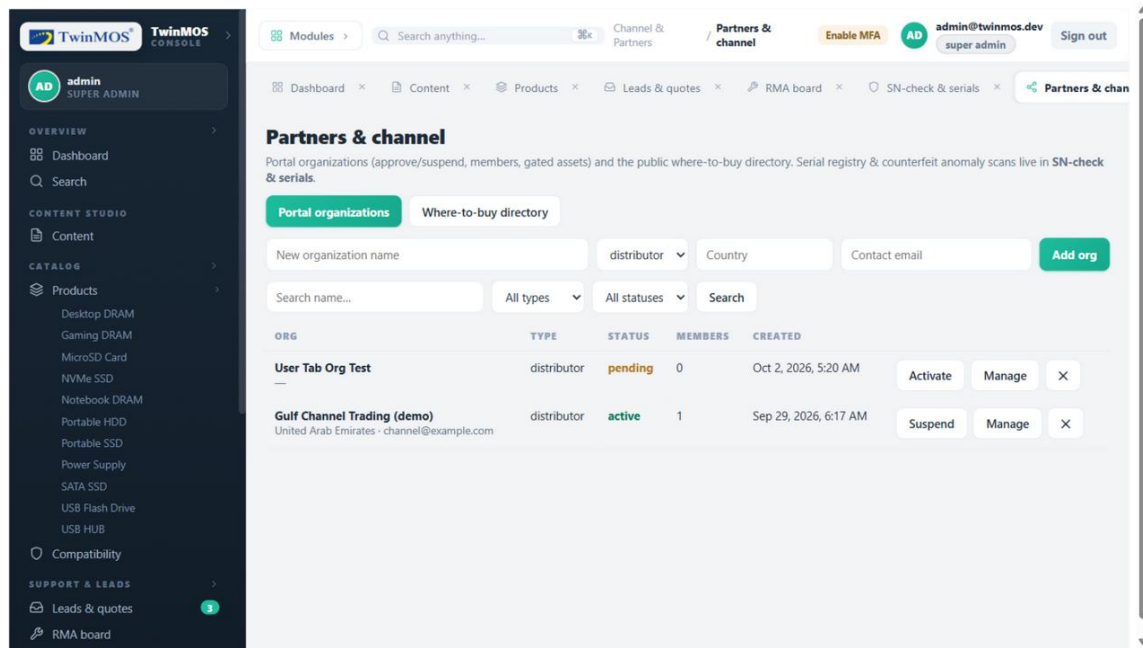


Exhibit 17. Partners & channel — portal organizations and the where-to-buy directory

Portal organizations

The first tab manages channel organizations (distributor / OEM / SI): create with country and contact email, activate or suspend, and open the detail drawer to add member accounts (staff sign-in emails), upload gated assets (price files, MDF documents, resources with per-type visibility) and delete when empty. Partners authenticate on the public site and see exactly the assets their organization is entitled to.

Where-to-Buy directory

The second tab is the public locator's source of truth: distributor entries with card title, distributor name, region (matching the site's filter chips), channel status (hub / authorized / expanding / seeking), cities, contact and card note — plus verified online marketplace listings. Search and region/status filters narrow the list; batch CSV import (dry-run + upsert by name+country) handles channel manifests. Published entries flow to the Where-to-Buy page through the content export.

5.10 Media Library (DAM)

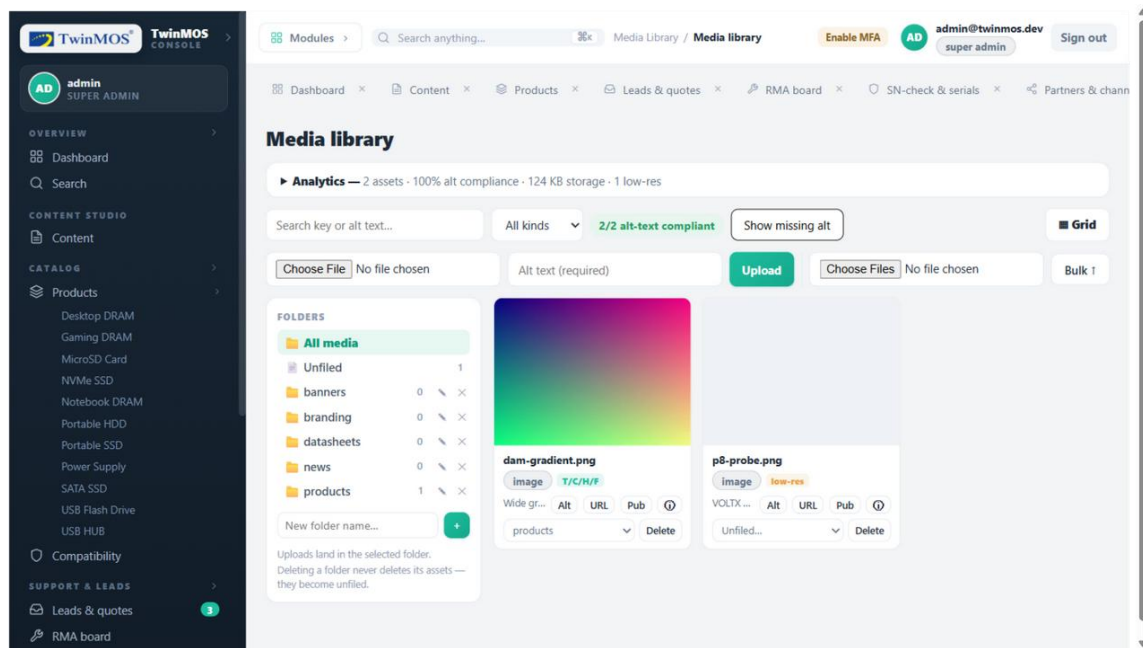


Exhibit 18. Media library — folder tree, analytics strip, search/kind filters and asset cards

1. Upload: pick a folder (the tree creates/renames/deletes; deleting never orphans assets), choose a file, provide the required **alt text** (accessibility), press Upload. Raster images automatically derive responsive variants (thumb/card/hero/full WebP + hero AVIF). **Bulk upload** accepts many files at once with alt text derived from filenames for later review.
2. Find: search across key and alt text; filter by kind or missing-alt. The analytics strip reports totals by kind, alt compliance, variant coverage and storage footprint.
3. Use: copy the admin URL (authenticated) or the public URL (site-usable) from any card, or open the **usage drawer** (the ⓘ button) to see exactly which products, articles and news posts reference the asset before deleting.
4. Govern: low-resolution masters (<1200px) are flagged per the brand guideline; deletes are admin-only and blocked while an asset is referenced.

5.11 Translation Studio

Nine locales are managed as string keys grouped by namespace with import/export for CAT-tool round-trips. The coverage heatmap (also shown in Content Studio's analytics) highlights key-exact coverage versus English so localization gaps are visible at a glance. Locale activation (which locales receive hreflang alternates) is a super_admin setting in Settings.

5.12 Users & Roles

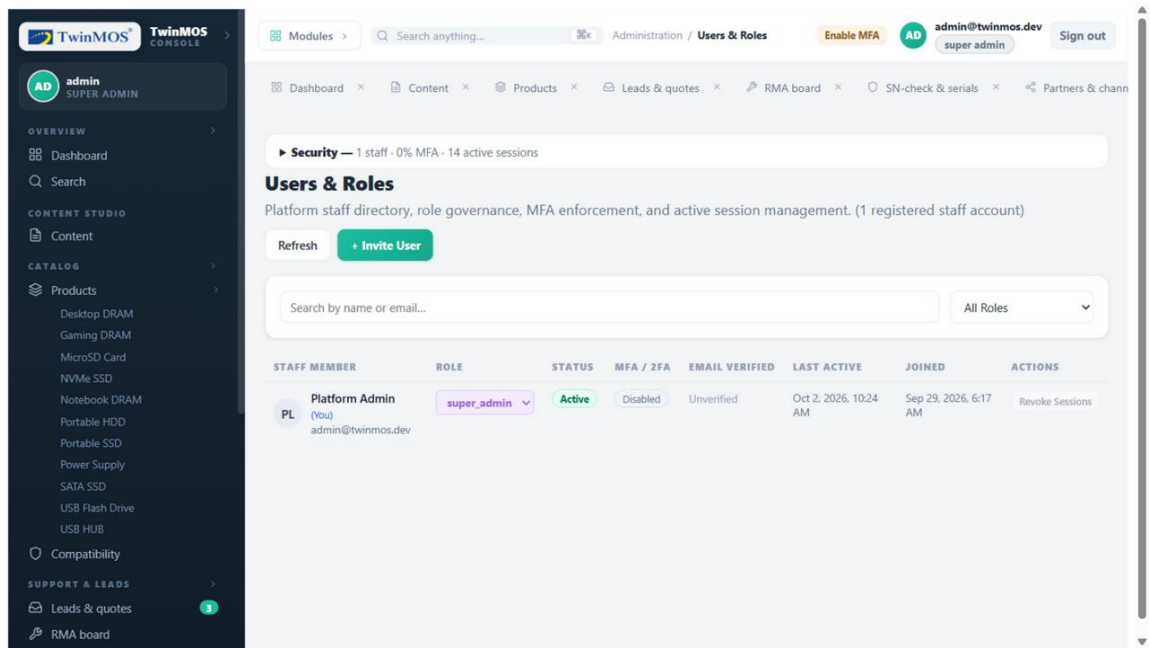


Exhibit 19. Users & Roles — directory with security analytics strip and per-user profile drawer

The super_admin-only module lists staff with search, role filter and pagination; invites new staff by email (role preset, optional initial password); changes roles (with a governance invariant preserving at least one active super_admin, race-safe under concurrent changes); revokes sessions (immediate logout); and suspends accounts with reason and optional expiry. The security analytics strip reports role distribution, MFA coverage, active session count and suspended accounts. Click any row for the per-user security profile: MFA state, live sessions and that account's last twenty audited actions.

5.13 Audit Log

Every mutation in every module lands here: actor, action, entity, entity id, request id, IP and timestamp, filterable by all of them. Retention is 12 months by default (configurable via environment). Use it to answer “who changed this price” or “what did this operator do” questions; the per-user view in Users & Roles is the same data scoped to one account.

5.14 Settings

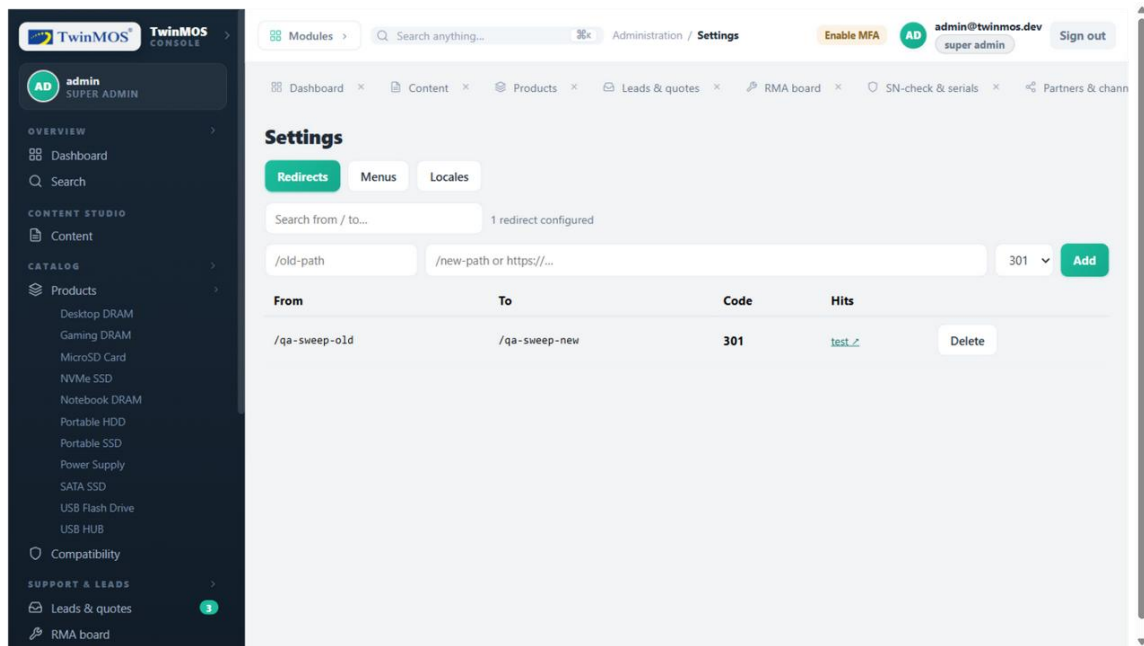


Exhibit 20. Settings — redirects manager, visual menu builder and locales

- **Redirects** — add from/to paths with 301/302/308 semantics; client-side validation (leading slash, http(s) targets, source $\neq$ destination), search, count and a live test link that opens the source path to watch it redirect.
- **Menus** — a visual builder editing the site menus document: per-group label/URL rows with add/remove/reorder, new-group creation and a JSON escape hatch for power users; changes save atomically with dirty-state tracking.
- **Locales** — the nine-locale registry with search and activation toggles (super_admin); active locales receive hreflang alternates in the site build.

6. Daily Operations Runbook (Webmaster)

6.1 Morning Check (10 minutes)

1. Open the **Dashboard**. Anything in the SLA risk queue? Open and triage those leads first.
2. Check the leads analytics strip: is the waiting-for-first-reply count growing? Answer new leads with **Reply to customer** — the five-minute rule is the single biggest conversion lever.
3. Glance at RMA board columns for submitted cases; run the serial check on any case whose card shows a serial.
4. Confirm media alt-text compliance is at 100% (dashboard widget links straight to the filtered media list).
5. Review the Content Studio Review Queue if authors submitted overnight.

6.2 Publishing Content to the Live Site

Publish inside the modules as described in §5; production builds export content automatically. If you operate a long-running staging or dev server and need changes visible immediately without a rebuild, re-run the exporter (see §7.6) and the merge layer picks them up on page refresh.

6.3 Weekly Rhythm

- Careers: close expiring postings (the list shows red closed-Xd-ago / amber closes-in-Xd warnings); review applications and reply.
- Serials: run the anomaly scan; investigate flagged serials with the drill-down; email the ops alert if the list is non-empty.
- Partners: review pending portal organizations; refresh the where-to-buy directory from channel updates (CSV import).
- Media: clear missing-alt items; flag low-resolution masters for replacement.
- Users: review the security strip — MFA coverage, active session count, suspended accounts — and offboard leavers (revoke sessions, then suspend).

6.4 Access Management Rules

- Every staff member gets an individual account with the minimum role that lets them work; shared logins are prohibited.
- TOTP two-factor is offered at sign-in and enforced by policy — assist enrollment rather than exempting accounts.
- Role changes are audited; the system always preserves at least one active super_admin, so demote carefully.
- On termination: revoke sessions first (immediate logout), then suspend with reason. The audit trail remains readable.

7. Cloud Deployment Guide (Software Engineer)

The codebase is production-ready and this runbook mirrors [docs/runbooks/DEPLOY.md](#) in the repository, which remains the source of record. The verified topology is: **Cloudflare (TLS, WAF) → VM nginx → static web + admin bundles on disk, /api/ reverse-proxied to a Node systemd service → PostgreSQL 16**. One origin keeps Better Auth cookies same-site.

7.1 Production Topology and Sizing

Component	Recommendation	Notes
VM	2 vCPU / 4 GB RAM / 40 GB SSD (web+api+db on one box to start)	Static assets are CDN-cached; the API is the only compute

Component	Recommendation	Notes
Node.js	v24 LTS (nodesource)	TypeScript executed natively — no build step for the API
PostgreSQL	16 (pgdg)	Enable WAL archiving for the 15-minute RPO (see DR.md)
nginx	Stable branch	Config shipped: deploy/nginx/twinmos.conf
Edge	Cloudflare proxied	TLS full-strict; Brotli; WAF; origin cert on nginx

7.2 Step 1 — Provision and Prepare the VM (once)

```
# user + directories
sudo useradd -r -s /usr/sbin/nologin twinmos
sudo mkdir -p /opt/twinmos/api /var/www/twinmos/web /var/www/twinmos/admin \
             /var/lib/twinmos/partner-files /var/lib/twinmos/media \
             /etc/twinmos /var/backups/twinmos
sudo chown -R twinmos:twinmos /var/lib/twinmos

# node 24 + postgres 16 + nginx (nodesource / pgdg / distro)
```

7.3 Step 2 — Database

```
sudo -u postgres createuser twinmos
sudo -u postgres createdb twinmos -O twinmos
# Enable WAL archiving (archive_command -> B2/backup volume) — see
docs/runbooks/DR.md
```

7.4 Step 3 — API Service

1. Ship the repository (git clone or rsync) to the VM, then install API dependencies: `cd apps/api && npm ci --omit=dev.`
2. Create the secrets file from the template — `sudo $EDITOR /etc/twinmos/api.env` copying `.env.production.example`, `chmod 600`. Fill every value: `DATABASE_URL`, `BETTER_AUTH_URL=https://www.twinmos.com`, `BETTER_AUTH_SECRET`, `ALLOWED_ORIGIN`, `TRUST_PROXY=1`, `RESEND_API_KEY`, `MAIL_FROM`, `FORMS_TO`, `TURNSTILE_SECRET_KEY`, upload dirs. All secrets come from the secret store — never commit filled copies.
3. Migrate and seed: `export $(cat /etc/twinmos/api.env | xargs) && npm run db:migrate && npm run db:seed` (seed on first boot only; rotate the seeded admin password at first sign-in).
4. Install and start the service: `sudo cp deploy/systemd/twinmos-api.service /etc/systemd/system/ && sudo systemctl daemon-reload && sudo systemctl enable --now twinmos-api.`

5. Verify: `curl -s http://127.0.0.1:8787/api/v1/health` must return `{"status":"ok"}`.

IMPORTANT — The systemd unit ships hardened (NoNewPrivileges, ProtectSystem=strict, ReadWritePaths limited to `/var/lib/twinmos` and the api data dir). Do not weaken these directives to work around permission errors — fix the path ownership instead.

7.5 Step 4 — Web + Admin Static Bundles

1. Build on a clean clone. The web build bakes environment into HTML: `PUBLIC_API_URL` must stay **unset** (empty = same-origin `/api/v1` behind nginx, exactly correct). Only the Turnstile site key needs setting — on Linux: `PUBLIC_TURNSTILE_SITE_KEY=0XXXXX npm run build`; or place it in `apps/web/.env` (gitignored). A guard aborts the production build if a stray `apps/web/.env` is present.

WARNING — Never assign `PUBLIC_API_URL=/api/v1` as a shell variable on Git Bash / Windows — MSYS rewrites the leading slash into a Windows path (`C:/Program Files/Git/api/v1`) which gets baked into the HTML. Build from a clean clone or use `apps/web/.env`.

1. `cd apps/web && npm ci && npm run build` (the prebuild step syncs vendored assets and runs the CMS content export automatically)
2. `cd ../admin && npm ci && npm run build`
3. `sudo rsync -a --delete apps/web/dist/ /var/www/twinmos/web/` and `sudo rsync -a --delete apps/admin/dist/ /var/www/twinmos/admin/`

7.6 Content Export and Rebuild Cycle

Published content reaches the static site through `cms-content.js`, regenerated by the web build's prebuild. To refresh content without a full redeploy, re-run the exporter and re-sync:

```
cd /opt/twinmos/repo
PGLITE_DATA='' DATABASE_URL=$DATABASE_URL node --experimental-strip-types \
  --no-warnings tooling/export-content.mjs # regenerates cms-content.js
sudo rsync -a apps/web/public/assets/js/cms-content.js
/var/www/twinmos/web/assets/js/
```

(In development the same command runs with `PGLITE_DATA` pointing at the embedded database and the API stopped — single-writer discipline; the command is documented in the repo README.)

7.7 Step 5 — nginx + Cloudflare

1. `sudo cp deploy/nginx/twinmos.conf /etc/nginx/sites-available/twinmos.conf` and symlink into sites-enabled; then `sudo nginx -t && sudo systemctl reload nginx`.
2. Point Cloudflare DNS at the VM (proxied). Edge settings: TLS Full (strict), Brotli, and an origin certificate installed at `/etc/nginx/certs/twinmos.pem`.

3. Confirm `TRUST_PROXY=1` is set in the api env (it ships in the template) so rate limiting reads cf-connecting-ip instead of the proxy address.

The shipped nginx config handles: one-origin routing (static web, /admin SPA with immutable hashed assets, /api/ proxy), the full security-header set, gzip, a 30 r/s coarse API guard, a 30 MB body cap, and the branded 404 page.

7.8 Step 6 — Backups and DR

```
sudo cp deploy/backup.sh /opt/twinmos/ && sudo chmod 700
/opt/twinmos/backup.sh
# cron (root): 15 2 * * * BACKUP_ROOT=/var/backups/twinmos DATABASE_URL=...
/opt/twinmos/backup.sh
# restore: deploy/restore.sh (production restore asks for the stamp; --drill
restores to a throwaway DB)
```

Targets: RTO 4 hours / RPO 15 minutes. DR.md documents the quarterly drill procedure; a local drill equivalent (tooling/dr-drill.mjs) is already recorded green in the phase evidence.

7.9 Step 7 — Cutover Checklist

- `curl https://www.twinmos.com/api/v1/health` returns ok.
- Sign in to /admin with the seeded admin and rotate the password immediately; create real staff accounts with MFA.
- Turnstile live on both sides (site key baked into the web build; secret in api.env); remove the dev bypass flag from the environment entirely.
- Resend live: submit the quote form and confirm the auto-reply arrives.
- Rate limits verified: rapid curl triggers 429 with Retry-After.
- Partner portal E2E on production (sign-in → gated download).
- SN-check on production with a registry serial → genuine verdict.
- First backup.sh run completed and one restore.sh --drill recorded in DR.md.
- Monitoring: uptime check on /health; log shipping for twinmos-api.
- ZAP baseline + load test re-run against production, recorded in the phase evidence.

7.10 Rollback

- Static bundles: keep the last three dated dist copies on the VM; redeploy the previous one.
- API: git checkout <previous tag> and restart twinmos-api.
- **Database migrations are forward-only** — if a migration must be undone, restore from backup with restore.sh.

7.11 Post-Deployment Monitoring

- Watch `/api/v1/health` (uptime alert) and systemd status for restarts (Restart=always with 3s backoff is configured).
- Watch the audit log volume — a sudden drop suggests submissions are failing upstream.
- Review Cloudflare analytics for WAF blocks on `/api/` (tune rules if legitimate traffic is challenged).
- Hypercare: the repository's HYPERCARE.md runbook defines a 14-day window with daily checks and exit criteria.

8. Testing and Quality Gates

8.1 Automated Suites

Gate	What it covers	Result at handover
Vitest (26 suites)	253 end-to-end + unit tests: auth/MFA lifecycle, RBAC races, CMS workflow, catalog, leads, RMA, directory, serials, media, settings, security headers	253/253 passed
Typecheck	TypeScript across api, admin, db, shared	Clean
Local OWASP-equivalent scan	tooling/owasp-local-scan.mjs — 14 check classes, 35 requests: headers, CORS, authz, traversal, SQLi/XSS reflection, verb tampering, rate limits, RFC 9457	NO FINDINGS
Parity gates (CI)	Playwright screenshot-diff vs the audited prototype	Passing (documented in phase evidence)
Accessibility	axe-core sweep with zero criticals	Passing

8.2 Running the Gates

```
cd
tw
in
ms
o
—
co
de
b
a
```

s
e
n
p
m
r
u
n
t
e
s
t

f
u
l
l
s
u
i
t
e
(
i
s
o
l
a
t
e
d
t
e
m
p
d
a
t
a
b
a
s
e
s
p
e
r
s
u
i
t
e
)
n
p
m
r
u
n
t

y
p
e
c
h
e
c
k

t
s
c
a
c
r
o
s
s
a
l
l
p
a
c
k
a
g
e
s
n
o
d
e
t
o
o
l
i
n
g
/
o
w
a
s
p
-
l
o
c
a
l
-
s
c
a
n
.
m
j
s

```
-
-
t
a
r
g
e
t
h
t
t
p
s
:
/
/
s
t
a
g
i
n
g
.
t
w
i
n
m
o
s
.
c
o
m
```

NOTE — The test suites use their own embedded databases (per-suite temp dirs) and are safe to run while the dev stack is up. The DR drill tool rebuilds a throwaway database from migrations + seed and probes a booted API — it is the fastest full-stack smoke test available.

9. Troubleshooting

9.1 Common Issues and Fixes

Symptom	Likely cause	Fix
Admin login rejected with 403 (not 401)	Auth origin not trusted — <code>BETTER_AUTH_TRUSTED_ORIGINS</code> missing the admin surface origin	Add the origin (e.g. <code>http://localhost:5173</code>) to <code>.env</code> and restart the API
Public forms / SN-check show unavailable (dev)	CORS: <code>ALLOWED_ORIGIN</code> unset so the browser blocks cross-origin API calls from <code>:4321</code>	Set <code>ALLOWED_ORIGIN=http://localhost:4321</code> in the dev <code>.env</code> ; restart

Symptom	Likely cause	Fix
API boots then queries fail with WASM abort (dev)	PGLite corrupted by concurrent access or force-kill	Restore from the data backup or npm run db:reset (dev data is reproducible); never run DB-writing tooling while the API is live
Health says ok but content queries 500	A migration was journaled but not applied (known embedded-migrator trap)	Verify new columns in information_schema; apply the migration SQL directly if missing (forward-only, idempotent statements)
Admin edits don't appear on the site	Content export not re-run (dev/staging)	Re-run tooling/export-content.mjs and refresh; production builds export automatically
429s from legitimate traffic	Rate buckets keyed on proxy IP — TRUST_PROXY unset	Set TRUST_PROXY=1 so cf-connecting-ip is honored
Turnstile never renders	PUBLIC_TURNSTILE_SITE_KEY not baked into the build	Rebuild the web bundle with the site key (see §7.5)
Uploads fail with size errors	nginx client_max_body_size vs MEDIA/PARTNER caps	nginx caps at 30 MB; env caps default 10 MB media / 25 MB partner files

9.2 Escalation Path

- Check systemd journal for twinmos-api first (journalctl -u twinmos-api -n 200).
- Reproduce API issues directly: curl the endpoint on 127.0.0.1:8787 to bypass edge and browser layers.
- Consult docs/runbooks/DR.md for data-layer incidents and HYPERCARE.md during the post-launch window.
- The audit log answers who changed what; the phase-evidence files document every gate's last green run.

10. Appendices

10.1 Appendix A — Environment Variables Reference (Production)

Variable	Required	Purpose
NODE_ENV	Yes	production — also hard-refuses PGLite and dev bypass flags
PORT	Yes	API listen port (8787)
DATABASE_URL	Yes	postgres://user:pass@host:5432/twinmos
BETTER_AUTH_URL	Yes	Public HTTPS origin; derives Secure cookies

Variable	Required	Purpose
BETTER_AUTH_SECRET	Yes	From secret store
ALLOWED_ORIGIN	Yes	CORS origin (same-origin in production)
TRUST_PROXY	Yes	Trust cf-connecting-ip for rate limiting
RESEND_API_KEY / MAIL_FROM / FORMS_TO	Yes	Transactional mail and routing (per-type overrides: RMA_TO, QUOTE_TO...)
TURNSTILE_SECRET_KEY	Yes	Server-side captcha verification
PUBLIC_TURNSTILE_SITE_KEY	Build-time	Baked into the web bundle to render the widget
MEDIA_DIR / PARTNER_FILES_DIR	Yes	Upload storage paths
MEDIA_MAX_BYTES / PARTNER_MAX_BYTES	No	Upload caps (defaults 10 MB / 25 MB)
AUDIT_RETENTION_DAYS	No	Audit retention (default 365)

WARNING — Never set `API_ALLOW_NO_TURNSTILE=1` or `API_DEV_AUTOLOGIN=1` in production. The API hard-refuses them under `NODE_ENV=production` regardless — keep them unset so misconfiguration is impossible.

10.2 Appendix B — API Route Map (Public + Auth + Admin)

Area	Routes
Public	POST /forms/:type (15 types) · GET /rma/:number · POST /sn-check · GET /i18n/:locale · GET /preview/:token · GET /media/:id/file (images) · GET /health
Auth	POST /auth/sign-in/email · /auth/sign-out · two-factor verify · GET /auth/get-session (Better Auth mount)
Partner portal	GET /partner/me · GET /partner/assets · GET /partner/assets/:id/download (member-gated)
Admin content	CRUD /admin/content/:entity · transitions · revisions · comments · preview tokens
Admin catalog	CRUD /admin/products (+variants, import/export CSV) · /admin/compatibility (+import) · /admin/taxonomy
Admin support	/admin/submissions (+reply, notes, CSV) · /admin/rma (+transition, assign, reply, serial-check)
Admin channel	/admin/partner-orgs (+members, assets, delete) · /admin/distributors (+import, export) · /admin/marketplace-listings
Admin brand protection	/admin/serials (+analytics, :serial drill-down/delete, import, export, anomalies) · /admin/sn-checks
Admin careers	/admin/job-postings · /admin/job-applications (+reply) · /admin/job-analytics

Area	Routes
Admin media	/admin/media (+analytics, :id/usage, variants) · /admin/media-folders
Admin governance	/admin/users (+invite, role, ban, revoke-sessions, :id/activity, analytics) · /admin/audit · /admin/settings · /admin/redirects · /admin/locales · /admin/analytics/* · /admin/staff-options

10.3 Appendix C — Role Permission Matrix

Capability	Viewer	Author	Editor	Admin	Super Admin
Read all modules	✓	✓	✓	✓	✓
Write content drafts	—	✓	✓	✓	✓
Publish / approve	—	—	✓	✓	✓
Catalog / directory / serials CRUD	—	—	✓	✓	✓
Partner org lifecycle	—	—	—	✓	✓
User management / locale activation	—	—	—	—	✓
Audit log read	—	—	✓	✓	✓

10.4 Appendix D — Glossary

Term	Definition
Bridge (export/merge)	The build-time pipeline that turns published DB rows into cms-content.js and merges them into the static site
DAM	Digital Asset Management — the media library with variants, folders and usage tracking
Deduped append	Merge-layer behavior: entries are matched by natural key and only genuinely new ones are added
PGLite	Embedded Postgres (WASM) used for dev/test; refused in production
QVL	Qualified Vendor List — the validated-device compatibility matrix
SLA risk queue	Dashboard list of open leads past (or near) their first-response deadline
SN-check	Public anti-counterfeit serial verification; also the module that manages the registry
Speed-to-lead	Time from lead arrival to first staff reply — the primary conversion KPI
Stakeholders: Webmaster / Engineer	Content-and-support operator vs deploy-and-operate engineer — the two audiences of this manual

10.5 Appendix E — File Manifest (Deployment-Critical)

Path	Purpose
deploy/nginx/twinmos.conf	One-origin edge config: static web, admin SPA, /api proxy, headers, caching
deploy/systemd/twinmos-api.service	Hardened service unit with EnvironmentFile
deploy/backup.sh / restore.sh	Nightly checksummed backups with rotation; stamped restore and --drill
.env.production.example	Every production variable documented; secrets only from the store
apps/api/scripts/{migrate,seed,reset}.ts	Database lifecycle scripts
tooling/export-content.mjs	The CMS export bridge (regenerates cms-content.js)
tooling/owasp-local-scan.mjs	Executable ZAP-baseline equivalent (14 check classes)
tooling/dr-drill.mjs	Full-stack DR drill: rebuild, verify counts, boot, probe
docs/runbooks/{DEPLOY,DR,HYPERCARE}.md	Operational runbooks of record